

Installation de la pile de simulation — Drone SLAM (proto mono-drone)

But de ce document : feuille de route d'installation à suivre **étape par étape** avec Claude Code. Chaque étape a un critère de validation. **Ne passe pas à l'étape suivante tant que la précédente n'est pas validée.** Objectif final : une démo simulée = vol dans Gazebo + caméra + retour capteur VL53 + carte qui se construit.

0. Prérequis machine

- ☐ **Ubuntu 24.04 LTS**, en installation **native** ou **dual-boot** (⚠ pas en machine virtuelle : Gazebo et Unity ont besoin du GPU).
- ☐ GPU avec pilotes à jour (NVIDIA de préférence si Unity/Isaac plus tard).
- ☐ ~50 Go d'espace libre, connexion internet.
- ☐ Git, build-essential, cmake, python3-pip installés.

```
bash

sudo apt update && sudo apt install -y git build-essential cmake python3-pip
```

Vue d'ensemble de la pile

Brique	Rôle	Version cible
Ubuntu	OS	24.04 LTS
ROS 2	orchestrateur (nœuds, topics)	Jazzy (LTS)
Gazebo	physique + capteurs simulés	Harmonic
ros_gz	pont Gazebo ↔ ROS 2	(paquet Jazzy)
PX4 SITL	contrôle de vol en simulation	gz_x500
Micro-XRCE-DDS Agent	pont PX4 ↔ ROS 2	eProsima
px4_msgs	définitions de messages PX4	branche = version PX4
Unity	rendu caméra photoréaliste	Unity 6 LTS
RTAB-Map	SLAM 3D (plus tard)	ros-jazzy-rtabmap-ros

Règle d'or d'architecture : le temps réel critique reste **en bas** (PX4), l'intelligence **en haut** (ROS 2). On ne remonte jamais la stabilisation dans des nœuds ROS.

1. ROS 2 Jazzy + outils de build

⚠ La méthode d'ajout du dépôt apt de ROS 2 a changé récemment (paquet `ros2-apt-source`). **Claude Code : suis la page officielle**

<https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html> pour l'ajout du dépôt/clé, puis :

```
bash

sudo apt update
sudo apt install -y ros-jazzy-desktop ros-dev-tools
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

- `ros-jazzy-desktop` → ROS 2 + **RViz2** + démos
- `ros-dev-tools` → **colcon**, **rosdep**, etc.

✓ Validation :

```
bash

ros2 run demo_nodes_cpp talker      # dans un terminal
ros2 run demo_nodes_cpp listener    # dans un autre → doit recevoir les message
```

2. Gazebo Harmonic + pont ros_gz

Depuis Jazzy, Gazebo est fourni via le dépôt ROS (paquets vendor). Une seule ligne :

```
bash

sudo apt install -y ros-jazzy-ros-gz
```

Installe Gazebo Harmonic et `ros_gz_bridge`.

✓ Validation :

```
bash

gz sim shapes.sdf      # la fenêtre Gazebo Harmonic s'ouvre avec des formes
```

3. PX4 SITL — le vol

```
bash
```

```
cd ~
git clone https://github.com/PX4/PX4-Autopilot.git --recursive
bash ./PX4-Autopilot/Tools/setup/ubuntu.sh
# (redémarrer la session si le script le demande)
cd ~/PX4-Autopilot
make px4_sitl gz_x500
```

`gz_x500` = quadricoptère de référence sous Gazebo Harmonic.

✓ **Validation** : Gazebo s'ouvre avec le drone `x500`. Dans la console PX4 (`pxh>`) :

```
commander takeoff      # le drone décolle
commander land
```

4. Pont PX4 ↔ ROS 2 (uXRCE-DDS)

4a. Micro-XRCE-DDS Agent

```
bash

cd ~
git clone https://github.com/eProsima/Micro-XRCE-DDS-Agent.git
cd Micro-XRCE-DDS-Agent
mkdir build && cd build
cmake ..
make
sudo make install
sudo ldconfig /usr/local/lib/
```

4b. Messages PX4 dans un workspace colcon

```
bash

mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src
git clone https://github.com/PX4/px4_msgs.git
git clone https://github.com/PX4/px4_ros_com.git # exemples (optionnel)
cd ~/ros2_ws
colcon build
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

⚠ `px4_msgs` doit correspondre à la **version de PX4** que tu as clonée (mêmes définitions de messages). Si tu as pris `main` pour PX4, prends `main` pour `px4_msgs`.

● POINT DE CONTRÔLE — ne pas continuer avant que ça marche

Trois terminaux :

```
bash

# Terminal 1 - l'agent
MicroXRCEAgent udp4 -p 8888

# Terminal 2 - le SITL (lance automatiquement le client uXRCE-DDS)
cd ~/PX4-Autopilot && make px4_sitl gz_x500

# Terminal 3 - côté ROS 2
ros2 topic list          # doit lister des topics /fmu/...
ros2 topic echo /fmu/out/vehicle_local_position # doit afficher la pose
```

✅ Si tu vois la pose du drone arriver dans le terminal ROS 2 → la colonne vertébrale tient. ❌ Si aucun topic /fmu/... : l'agent n'est pas lancé, ou mauvaise version, ou pare-feu/port. NE PAS attaquer Unity avant d'avoir réglé ça.

Optionnel très pratique : QGroundControl (AppImage, <https://qgroundcontrol.com>) pour armer/décoller à la souris pendant le debug.

5. Unity + pont ROS (rendu caméra)

⚠️ Maillon le moins entretenu de la chaîne : prévois du réglage réseau (IP/port). À ne faire **qu'après** le point de contrôle validé.

- ☐ Installer **Unity Hub**, puis **Unity 6 (LTS)** via le Hub (éditeur Linux).
- ☐ Créer un projet en template **URP** (3D).
- ☐ Dans Unity → Package Manager → *Add package from git URL* : **ROS-TCP-Connector** (<https://github.com/Unity-Technologies/ROS-TCP-Connector.git> path=/com.unity.robotics.ros-tcp-connector))
- ☐ Côté ROS, cloner et compiler **ROS-TCP-Endpoint** dans le workspace :

```
bash

cd ~/ros2_ws/src
git clone -b main-ros2 https://github.com/Unity-Technologies/ROS-TCP-Endpoint.git
cd ~/ros2_ws && colcon build
```

- ☐ Lancer l'endpoint : `ros2 run ros_tcp_endpoint default_server_endpoint --ros-args -p ROS_IP:=127.0.0.1`
- ☐ Dans Unity : Robotics → ROS Settings → renseigner la même IP/port.

✅ **Validation** : un message publié depuis Unity apparaît via `ros2 topic echo`, et inversement.

6. SLAM (plus tard — pas pour la démo minimale)

```
bash

sudo apt install -y ros-jazzy-rtabmap-ros
# optionnel, OctoMap explicite :
sudo apt install -y ros-jazzy-octomap-server
```

RTAB-Map consomme profondeur + IMU et produit nuage de points + OctoMap.

Pièges connus (à donner à Claude Code si ça coince)

1. **Tutos en Humble** : 90 % des guides sont écrits pour ROS 2 Humble / Ubuntu 22.04. Toi = **Jazzy / 24.04**. Remplace `humble` → `jazzy` partout.
 2. **Agent non lancé** : en simu, PX4 démarre le *client* uXRCE-DDS, mais l'**agent**, c'est à toi de le lancer. Sans lui, aucun topic ROS.
 3. **Sourcing** : dans chaque nouveau terminal ROS → `source` `/opt/ros/jazzy/setup.bash` puis `source ~/ros2_ws/install/setup.bash` (déjà dans `.bashrc` si tu as suivi).
 4. **Version px4_msgs ≠ PX4** → messages illisibles. Aligne les branches.
 5. **VM / GPU** : si Gazebo rame ou écran noir, c'est souvent le GPU/pilote, pas ROS.
 6. **Unity ↔ ROS** : problème d'IP/port dans 9 cas sur 10.
-

Ordre de lancement quotidien (une fois installé)

1. Terminal 1 : `MicroXRCEAgent udp4 -p 8888`
 2. Terminal 2 : `cd ~/PX4-Autopilot && make px4_sitl gz_x500`
 3. Terminal 3 : `ros2 run ros_tcp_endpoint default_server_endpoint ...` (si Unity)
 4. Terminal 4 : `tes nœuds ROS 2`
 5. Unity : `Play`
-

Comment bosser avec Claude Code sur ce document

- **Une étape à la fois**. Demande-lui de faire/valider l'étape N, pas « installe toute la pile ».
- **Coche au fur et à mesure** et vérifie chaque ✅ Validation avant d'avancer.
- **Donne-lui les vrais logs d'erreur** (copier-coller complet) : c'est là qu'il est le plus efficace.
- **Sécurise d'abord les étapes 1 → 4 + le point de contrôle** avant de toucher à Unity et au SLAM.

- **Comprends ce qui se passe** à chaque étape : le jour où ça casse (ça cassera), tu dois savoir où regarder.

Jalons de progression

- ☐ Jalon 1 : ROS 2 talker/listener OK
- ☐ Jalon 2 : Gazebo Harmonic s'ouvre
- ☐ Jalon 3 : PX4 SITL vole (takeoff/land)
- ☐ Jalon 4 : ● pose du drone visible dans un topic ROS 2
- ☐ Jalon 5 : Unity ↔ ROS échangent un message
- ☐ Jalon 6 : nœud minimal qui lit la pose et envoie un décollage
- ☐ Jalon 7 : caméra + grille VL53 (raycasts) + carte qui se construit